API Payload Decryption Documentation

Overview

This document describes the decryption process for encrypted API payloads. The implementation uses AES encryption with SHA-256 checksum verification to ensure data integrity.

Encryption Specifications

Algorithm Details

- **Encryption Algorithm**: AES
- **Mode**: ECB
- **Padding**: PKCS5Padding
- **Key Format**: Base64-encoded string
- **Payload Encoding**: Base64
- **Checksum Algorithm**: SHA-256
- **Text Encoding**: UTF-8

Security Model

The encrypted payload contains a SHA-256 checksum of the original payload. This checksum is used for integrity verification during decryption.

Implementation Guide

Prerequisites

- Base64 decoder/encoder
- AES decryption capability
- SHA-256 hashing function
- UTF-8 text encoding support

Step-by-Step Decryption Process

1. **Decode the Encryption Key**

```
keyBytes = base64Decode(encryptionKey)
```

3. **Initialize AES Cipher**

```
cipher = initAESCipher(keyBytes, DECRYPT_MODE)
```

5. **Decrypt the Payload**

```
encryptedBytes = base64Decode(encryptedPayload)
decryptedBytes = cipher.decrypt(encryptedBytes)
decryptedString = utf8ToString(decryptedBytes)
```

9. **Verify Integrity**

```
expectedChecksum = sha256Hash(originalPayload)
expectedChecksumB64 = base64Encode(expectedChecksum)

if (expectedChecksumB64 != decryptedString) {
```

14. throw SecurityException("Checksum verification failed")
15. }
16. **Return Original Payload**
17. return originalPayload

Code Examples

Java Implementation

```
public String decryptPayload(String encryptedPayload, String
originalPayload) throws Exception {
    // Convert key
    byte[] keyBytes = Base64.getDecoder().decode(encryptionKey);
    SecretKeySpec key = new SecretKeySpec(keyBytes, "AES");

    // Decrypt
    Cipher cipher = Cipher.getInstance("AES");
    cipher.init(Cipher.DECRYPT_MODE, key);
    byte[] decrypted =
cipher.doFinal(Base64.getDecoder().decode(encryptedPayload));
    String decryptedStr = new String(decrypted, StandardCharsets.UTF_8);

    // Verify checksum
    MessageDigest digest = MessageDigest.getInstance("SHA-256");
    byte[] calculatedChecksum =
digest.digest(originalPayload.getBytes(StandardCharsets.UTF_8));
    String calculatedChecksumStr =
Base64.getEncoder().encodeToString(calculatedChecksum);

    if (!calculatedChecksumStr.equals(decryptedStr)) {
        throw new SecurityException("Checksum verification failed - payload
may be corrupted");
    }

    return originalPayload;
}
```

Python Implementation

```
import base64
import hashlib
from Crypto.Cipher import AES
from Crypto.Util.Padding import unpad

def decrypt_payload(encrypted_payload, original_payload, encryption_key):
    # Decode the key
    key_bytes = base64.b64decode(encryption_key)

    # Decrypt
    cipher = AES.new(key_bytes, AES.MODE_ECB)
    encrypted_bytes = base64.b64decode(encrypted_payload)
    decrypted_bytes = cipher.decrypt(encrypted_bytes)

    # Remove padding and decode
    decrypted_str = decrypted_bytes.decode('utf-8').rstrip('\x00')

    # Verify checksum
```

```
    calculated_checksum = hashlib.sha256(original_payload.encode('utf-
8')).digest()
    calculated_checksum_str =
base64.b64encode(calculated_checksum).decode('utf-8')

    if calculated_checksum_str != decrypted_str:
        raise SecurityError("Checksum verification failed - payload may be
corrupted")

    return original_payload
```

Node.js Implementation

```
const crypto = require('crypto');

function decryptPayload(encryptedPayload, originalPayload, encryptionKey) {
    // Decode the key
    const keyBytes = Buffer.from(encryptionKey, 'base64');

    // Decrypt
    const decipher = crypto.createDecipher('aes-128-ecb', keyBytes);
    const encryptedBytes = Buffer.from(encryptedPayload, 'base64');
    let decrypted = decipher.update(encryptedBytes);
    decrypted = Buffer.concat([decrypted, decipher.final()]);
    const decryptedStr = decrypted.toString('utf8');

    // Verify checksum
    const calculatedChecksum = crypto.createHash('sha256')
        .update(originalPayload, 'utf8')
        .digest('base64');

    if (calculatedChecksum !== decryptedStr) {
        throw new Error('Checksum verification failed - payload may be
corrupted');
    }

    return originalPayload;
}
```

API Contract

Input Parameters

- `encryptedPayload` (string): Base64-encoded encrypted checksum
- `originalPayload` (string): The original unencrypted payload data
- `encryptionKey` (string): Base64-encoded AES encryption key

Output

- Returns the `originalPayload` string if decryption and verification succeed
- Throws `SecurityException` if checksum verification fails

Error Handling

- **Checksum Mismatch**: Indicates payload corruption or tampering

- **Decryption Failure**: Invalid key or corrupted encrypted data
- **Encoding Errors**: Invalid Base64 input

Testing

Test Vector Example

```
encryptionKey: "MTIzNDU2Nzg5MDEyMzQ1Ng=="
originalPayload: "Hello, World!"
expectedChecksum:
"dffd6021bb2bd5b0af676290809ec3a53191dd81c7f70a4b28688a362182986f"
expectedChecksumB64: "3/1gIbsr1bCvZ2KQgJ7DpTGR3YHH9wpLKGiKNiGCmG8="
```

Support

This documentation should provide all necessary information for implementation. The decryption process is standardized and should work consistently across different programming languages and platforms.